Related work

Sep 27, 2026 · @Dimitrios Stamatios Bouras

After about 30 papers, one claim survives: a certified bound on the chance that a clean verdict of an unsound taint analysis hides a missed leak, built from calibrated priors over missing library behaviour and checked on held-out apps. Every other piece has prior art. Receipts as minimal assumption sets go back to Bastani (2015), Zhu (2013) and Christakis (2015). Probabilistic spec inference goes back to Merlin (2009). LLM facts fed to Soufflé appear in NeuroLog (2026). Asking an LLM about local flow edges, chosen by downstream value, is Zhang et al. (OOPSLA 2026), aimed at false alarms.

Ingredient	Prior art	New in yours?
Minimal set of assumptions behind a verdict	Bastani 2015; Zhu 2013; Christakis 2015	No
Probabilities on library specs	Merlin 2009	No
LLM output as analysis facts	IRIS 2025; NeuroLog 2026; SemTaint 2026	No
LLM queries on local facts, picked by value	Zhang et al. OOPSLA 2026 (false alarms)	Direction only (missed leaks)
Risk bound per clean verdict	None found	Yes
Guarantee checked on held-out apps	None found for static analysis	Yes
Holes to check chosen by receipts, yielding missed leaks and verified models	Atlas 2018 checks specs by running them, without this targeting	Partly

The biggest threat is Xin Zhang's group: the OOPSLA 2026 machinery points at flow edges already. The one paper I could not read is listed at the end of Residual risk from testing.

Closest work: minimal assumptions behind a verdict

The receipt idea (the smallest set of assumptions about missing code that a verdict rests on) is established; the Aiken and Dillig groups had it for Android taint by 2015. None of them prices the assumptions or bounds the error, and all send the assumptions to a human.

Work	What it computes	How it differs from ours
Bastani, Anand, Aiken, OOPSLA 2015	Min-cuts in a source–sink flow graph; each cut is a set of statements assumed unreachable; a developer confirms with tests; iterate	Same cut structure. Unweighted, human-checked, no error rate. Our receipt is their cut with prices.
Bastani, Anand, Aiken, POPL 2015	Infers Android library flow and alias specs by CFL reachability; flows needing fewer assumed specs are ranked as more likely real; an auditor confirms	The leak-finding mirror of ours, ranking by count of assumptions. No probabilities.
Zhu, Dillig, Dillig, APLAS 2013	Abduces the library specs needed to verify a source–sink property on Android apps	Minimal specs for verification, exactly our "what does clean rest on". No pricing, no guarantee.
Dillig, Dillig, Aiken, PLDI 2012	Abduction finds minimal proof obligations that would confirm or refute a report; asks the user	Same idea for general reports; human as oracle.
Zhou et al., OOPSLA 2021 (Elrond)	Data-driven abduction of library specs sufficient to verify a client, refined by SMT counterexamples	Learns specs from tests instead of asking a human; OCaml data structures, not taint; no error rate.

What you can claim over this line: replacing the human auditor with a calibrated prior, turning the cut into a risk bound, and certifying that bound on held-out apps.

Library spec inference for taint

This line fills holes; it never says how wrong the filling might be. Merlin is the one exception that uses probabilities, but only to pick specs, not to bound a verdict's error.

Work	What it infers	Uncertainty?
Merlin, PLDI 2009	Sources, sinks, sanitizers from a propagation graph, via a factor graph	Marginal probabilities, thresholded into specs; then treated as fact
StubDroid, ICSE 2016	Precise flow summaries for Android framework methods, from their code	None; sound up to its analysis
Seldon, PLDI 2019	Source/sink/sanitizer roles from big code by linear optimization	Reports 67% estimated precision; no per-spec confidence used downstream
Atlas, PLDI 2018	Points-to specs for Java library methods by running them (active learning)	None; tests as oracle. Our oracle 2 is close to this.
IRIS, ICLR 2025	LLM infers taint specs for library APIs, fed to CodeQL	LLM answers taken as fact
SemTaint, 2026	Multi-agent LLM extraction of sources, sinks, call edges and library flow summaries for npm packages, fed to CodeQL	No confidence reported

Atlas is worth citing next to closing: it already executes library methods to learn specs. Your difference is which methods to test (the ones receipts depend on) and what the result buys (a certified verdict).

LLMs inside taint and static analysis

These systems use LLM output to find more bugs. None attaches a risk to a clean verdict, and none reports which LLM claims a result depends on in a checkable form.

Work	Role of the LLM	Relation to ours
Towards LLM-Enhanced Android Taint Analysis, 2026	An agent explores decompiled apps and traces flows itself; majority vote over 3–5 runs	Replaces the analysis instead of pricing its holes; DroidBench plus 5 real apps; no guarantee

Work	Role of the LLM	Relation to ours
AndroByte, 2025	LLM summarizes bytecode to build dataflow call graphs for privacy analysis	LLM summaries as fact
NeuroLog, 2026	LLM extracts typed dataflow facts per function; Soufflé rules compose them into findings	Closest architecture (LLM facts + Soufflé). Facts are unweighted and the goal is finding bugs, not certifying their absence.

NeuroLog needs a sentence in the paper: same stack, opposite question.

Probabilistic and Bayesian program analysis

This is the most active competing line, and it is next door in Xin Zhang's PKU group. Every paper here works on false alarms; none targets missed bugs from unsoundness. Their OOPSLA 2026 paper is the closest in mechanism to your closing loop.

Work	What it does	Side
Bingo, PLDI 2018	Turns Datalog derivation graphs into a Bayesian network; user feedback on alarms updates the ranking	False alarms
Zhang et al., OOPSLA 2026	An LLM judges small intermediate facts (alias or flow edges); the Bayesian analysis picks which facts to ask about to maximize downstream pruning	False alarms. Same shape as your closing loop (ask about local facts, chosen by downstream value), opposite direction.
Shi, Jin, Zhang, PLDI 2026	Probability that an abstract interpreter is locally complete, i.e. how likely an alarm is imprecision	False alarms; assumes the analysis is sound

Work	What it does	Side
PPProbe, 2026	Finds minimal sets of mutually inconsistent outputs in probabilistic Datalog to prune alarms	False alarms
BinLLM, FITEE 2025	LLM-derived rules added to the Bingo-style probabilistic model	False alarms
Fuzzing guided by Bayesian program analysis, POPL 2026	Uses Bayesian analysis posteriors to steer a fuzzer	Bug finding

Two more differences to state: they use posteriors (no guarantee on error), and their probabilities attach to rules or derived facts, not to missing library behaviour. The risk is real: the OOPSLA 2026 machinery already asks an LLM about flow edges, and pointing it at holes is a small step for them.

Soundiness and measuring unsoundness

This line measures unsoundness in aggregate or makes assumptions explicit per method. Christakis et al. are the closest: explicit assumptions at each point, checked by tests. Nobody gives a per-verdict bound.

Work	What it does	Relation to ours
Livshits et al., "In Defense of Soundiness", CACM 2015	Names the practice and asks tools to document their unsound choices	The motivation; cite in the intro
Christakis, Müller, Wüstholz, VMCAI 2015	Instruments Clousot's 11 kinds of unsound choices as `assumed P` statements; tests show 2–26% of them are violated at run time	Explicit assumptions plus testing, like receipts plus oracle 2. No probabilities, no bound, method-level.

Work	What it does	Relation to ours
Christakis, Müller, Wüstholz, ICSE 2016	Steers dynamic symbolic execution toward executions the static analyzer did not verify	Assumptions direct testing effort, as in your closing loop
Samhi et al., ISSTA 2024	13 Android static tools miss on average 61% of methods executed dynamically, across 1,000 apps; external frameworks are the top cause	Aggregate evidence that library holes matter on Android. Good motivation citation.
Detecting call graph unsoundness without ground truth, 2026	Metamorphic testing over precision orders of call-graph variants	Finds that unsoundness exists, not which verdicts it affects
Qiu et al., TSE 2021	Compares FlowDroid, Amandroid, DroidSafe on the same apps	Library modelling as a source of disagreement

Uncertainty and statistical guarantees

The statistics you use are standard; the new part is where you apply them. I found no paper that puts a conformal or PAC guarantee on the verdicts of a static analysis, including in 2026 preprints. Cite these as the tools you build on.

Work	What it guarantees	How you use it
Learn then Test, Angelopoulos et al.	Calibrate a threshold so any chosen risk is controlled with high probability	Your per-app threshold with 90% confidence of ⩽ 10% risk is this framework; say so
Mohri, Hashimoto, ICML 2024	Back off an LLM's claims until the remaining output is correct with a chosen probability	Close analogue: you back off clearances until the rest are safe with a chosen probability
Khakhar et al., ICML 2023	PAC prediction sets for code LLMs (Bastani's group again)	Same group as the closest cut work; they have both halves, but have not combined them

Work	What it guarantees	How you use it
Spiess et al., ICSE 2025	Measures calibration of code LLMs; they are often poorly calibrated	Motivates RQ1 and a calibrated pricer; matches your 0.72 and overconfidence finding for the cheap model
PAC learning-based verification, ICSE 2016	PAC guarantees for learned models in verification	Earlier use of PAC in SE verification; different object

Datalog provenance and causality

Database theory already says when your receipt is exact and when it is hard: minimum cut is exact for reachability-shaped rules and NP-hard for general joins. This is the formal section an OOPSLA version needs, and it should be cited rather than re-derived.

Work	Result	What it gives you
Meliou et al., VLDB 2010	Responsibility (the smallest set of tuples whose removal changes an answer) reduces to max-flow for linear queries; a dichotomy shows it is NP-hard otherwise	Receipt = weighted responsibility over hole facts. Exact by min-cut when the taint rules are linear; otherwise a bound or NP-hard.
Causality and minimal supports in recursive Datalog, 2026	For reachability, minimal supports are simple paths and deletion robustness equals the minimum edge cut; general transitive closure has unbounded supports	Direct justification for min-cut on the flow graph
The complexity of why-provenance for Datalog, 2024	Complexity of computing why-provenance for Datalog	Background for the general (non-linear) case
Zhao, Subotić, Scholz, TOPLAS 2020	Scalable provenance (proof trees) in Soufflé	Engineering route to extract derivations if you move beyond pure reachability

What to write: taint propagation over the flow graph is linear, so the receipt is exact; with joins (for example, field-sensitive aliasing) the cut may not be the cheapest receipt, so the bound gets looser, but it stays valid as long as the graph over-approximates the derivations.

Residual risk from testing

Böhme's line is the testing-side twin of your idea: a number for what has not been found yet. It is about inputs not yet run, not about code an analysis skipped.

Work	Estimate	Relation to ours
Lee, Böhme, ICSE 2026	Probability the next test input reveals a bug, accounting for dependencies between coverage elements	Handles dependence explicitly, as your per-app calibration does; frame yours as the static counterpart
Böhme, Liyanage, Wüstholz, FSE 2021	Residual risk after a fuzzing campaign finds nothing	Origin of the "residual risk" framing; borrow the term
Statistical reachability analysis, FSE 2023	Estimates reachability probabilities from executions	Statistical, execution-based

One more to read yourself: Soundness Checking of Taint Flow Models (arXiv, Sep 2026). The title suggests it checks library taint models for soundness, which overlaps with your closing loop. I could not open it.

Sources

All links are in the tables above; each table's first column links the paper or page read. Livshits et al., "In Defense of Soundiness" (CACM 2015) is cited from memory and should be checked.